Unit 1: Introduction to Problem Solving and Computing Concepts

This unit establishes the foundation for the course by defining what constitutes a "problem" in the context of computing and exploring the systematic ways we approach them.

1.1 Core Concepts of Computing

Before solving problems with a computer, we must understand the computer's role. A computer is essentially a **data processor** that follows specific instructions to transform **Input** (raw data) into **Output** (meaningful information). Problem solving in this context is the process of designing those instructions.

1.2 Defining a "Problem"

In computing, a **problem** is a discrepancy between a current state and a desired goal state.

- **Problem Identification:** This is the first and most critical step. It involves clearly defining what needs to be achieved and identifying the constraints (limitations like time, memory, or hardware).

1.3 Types of Problems

Problems are generally categorized into two types based on the complexity of the solution required:

1. **Routine Problems:**
    - These are problems that can be solved using established methods or formulas.
    - They require little creative leap; once the "type" of problem is identified, the procedure is clear.
    - *Example:* Calculating the area of a circle or sorting a list of names alphabetically.
2. **Non-Routine Problems:**
    - These are novel or complex problems where a standard procedure is not immediately obvious.

    - They require higher-order thinking, creativity, and often a combination of different solution techniques.
    - *Example:* Designing an AI to detect hate speech in multiple languages or optimizing a city's traffic flow.

1.4 Solvable vs. Unsolvable Problems

Not every problem defined by a human can be solved by a computer.

- **Solvable Problems:** There exists an algorithm (a step-by-step procedure) that can solve the problem in a finite amount of time.
- **Unsolvable (Undecidable) Problems:** These are problems for which it is mathematically proven that no algorithm can be constructed to always lead to a correct "yes" or "no" answer.
    - *Classic Example:* **The Halting Problem**—it is impossible to write a program that can determine, for any given program and input, whether that program will eventually stop or run forever.

1.5 Introduction to Algorithms and Heuristics

There are two primary ways to approach a solution:

- **Algorithms:** A logical, step-by-step procedure that **guarantees** a correct solution if followed correctly. Computers primarily rely on algorithms.
- **Heuristics:** "Rules of thumb" or mental shortcuts. They do not guarantee an optimal or perfect solution but are useful for finding a "good enough" solution quickly, especially when an algorithmic approach is too slow or complex.

While both are used to reach a goal, they differ significantly in their approach and reliability.

1.5.1 The Algorithmic Approach

An algorithm is a finite set of unambiguous instructions that, when followed, produces a specific result.

- **Characteristics:**
    - **Determinism:** Given the same input, it always produces the same output.
    - **Finiteness:** It must terminate after a specific number of steps.
    - **Precision:** Each step is clearly defined.
- **Computing Context:** Computers are "algorithmic machines." They require the rigid structure of an algorithm to function.

1.5.2 The Heuristic Approach

Heuristics are often described as "mental shortcuts" or "rules of thumb." They are practical methods used when an optimal solution is either impossible to find or computationally too expensive.

- **Characteristics:**
    - **Speed:** Much faster than searching every possible solution.
    - **Satisficing:** They aim for a "good enough" solution rather than the "best" one.
    - **Trial-based:** Often relies on past experience or educated guesses.
- **Computing Context:** Used heavily in **Artificial Intelligence**, search engines, and complex game design (like Chess) where calculating every possible move would take too long.

Feature	**Algorithm**	**Heuristic**
Goal	Guaranteed Accuracy	Speed and Efficiency
Reliability	100% Correct (if logic is sound)	May lead to errors or sub-optimal results
Use Case	Mathematical calculations, Sorting	Large-scale data search, AI, daily decisions

1.6 Summary of Unit 1

At this point, students should understand that:

1. Computers solve problems by processing data through instructions.
2. Problems can be **routine** (procedural) or **non-routine** (complex/creative).
3. Some problems are **unsolvable** by any machine.
4. We choose between **Algorithms** (accuracy) and **Heuristics** (speed) based on the problem's nature.